

TTFL: Trust-Flow-Driven Trusted Federated Learning for Edge Computing

Jiakai Zhou, Chao Bu, Shuo Zhang, Xinyu Liu

School of Computer Science and Engineering

Tianjin University of Technology, Tianjin, China

Email: wllsp0@stud.tjut.edu.cn, bc_0722@163.com

Abstract—Federated learning at the edge must distinguish benign data heterogeneity from harmful updates without allowing past contributions to conceal current risk. TTFL combines authenticated execution reports, server-side content audits, and separate historical-utility and short-term-risk streams. The streams control reversible participation states and layer-wise aggregation. Server-supported recovery audits remove the need for eligible peers during quarantine, while attainable score gates, block clipping, and survivor normalization specify how admitted updates contribute. Evaluation has two scopes: a CIFAR-10 training study of the original implementation under mixed attacks, and executable checks of the explicit controller rules. The training study reports accuracy, attack outcomes, client trajectories, and aggregation comparisons; its results are not measurements of a retrained controller. In a constructed test with twenty initially quarantined clients, aggregation resumes at the fourth successful audit and all clients return to NORMAL at the sixth; a matched peer-dependent audit remains frozen. The checks establish conditional recovery and aggregation behavior. Matched training is still needed to quantify the effect of the controller refinements on model quality.

Index Terms—Federated learning, edge computing, trusted execution environment, risk modeling, layered aggregation, backdoor defense

1. Introduction

FL enables edge devices to train a shared model through parameter exchange while keeping their raw samples local [1], [2]. In an open edge deployment, however, the server must assess both the execution that produced an update and its effect on the global model. This assessment becomes difficult under Non-IID data: a legitimate long-tail client may repeatedly deviate from a reference direction, whereas an attacker may build a favorable record before submitting a harmful update. A single reputation score makes these cases difficult to separate. Tightening its threshold can remove useful heterogeneous contributions, while allowing past performance to compensate for current anomalies can delay the response to an attack. The resulting challenge is to preserve evidence of long-term contribution without allowing that evidence to conceal short-term risk.

Existing defenses use geometric, statistical, reference-based, or temporal evidence to flag unusual updates [3], [4], [5], [6], [7]. Backdoor-oriented methods additionally examine model behavior, learned representations, clusters, and collusion [8], [9], [10], [11]. Such tests say little about how local training was executed, and the same tests can interpret Non-IID drift as an attack. TEE-assisted FL contributes a hardware-rooted execution boundary [12], [13], [14], [15]; admission based on that boundary still leaves the semantic safety of an admitted update unresolved.

The proposed *Trust Flow* policy maintains historical utility and short-term risk as separate states. Historical utility records contribution, while current risk can restrict an update despite a favorable earlier record. These states connect execution admission, content auditing, and layer-wise aggregation. A recovery path must also remain usable when the eligible peer population is empty, and layer thresholds must respect the range of the credibility score. The contributions are:

- 1) An evidence-driven dual-stream framework connects authenticated reports, current-update audits, and persistent participation states, separating recoverable restrictions from permanent server-side rejection.
- 2) Server-supported recovery and attainable layer gates make the control rules explicit. Updated states determine eligibility; clipped survivors receive sample-size- and credibility-weighted contributions.
- 3) A training study characterizes the original framework on CIFAR-10, while executable boundary tests verify the specified recovery, state, weighting, and empty-set behavior. The two evaluations retain separate implementation and evidence scopes.

2. Background and Related Work

Federated learning in edge computing. Local training and server aggregation allow clients to collaborate without transferring raw samples, but leave the server dependent on the evidence accompanying each update. Heterogeneous data distributions complicate that assessment because a useful long-tail contribution may differ from the majority. Execution provenance and update quality consequently serve

different purposes: the former establishes how an update was produced, while the latter determines how it should contribute to learning.

Poisoning and backdoor attacks. Sign flipping and gradient scaling disrupt optimization by changing update directions or magnitudes. Trigger, clean-label, and semantic backdoors instead seek targeted errors while preserving plausible main-task behavior, so their updates can remain close to an expected direction [16], [17]. Across rounds, an attacker may also accumulate a favorable history before injecting a harmful update, making current behavior and past contribution potentially conflicting sources of evidence.

Active defenses and their limits. Krum and Trimmed Mean remove geometric outliers [3], [4], FLTrust weights updates against a clean direction [5], and FoolsGold and FLDetector use cross-client or cross-round evidence [6], [7]. DeepSight, FLAME, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA inspect representations, backdoor behavior, collusion, or privacy-preserving updates [8], [9], [10], [11], [18], [19], [20]. TMT-FL and related systems connect execution integrity to model training [15]. These approaches observe different evidence spaces. Trust Flow places an interface between them: attestation determines execution eligibility, privacy-aware checks expose bounded statistics, content probes inspect the current update, and the two temporal streams decide future privileges. One observation can therefore alter a layer weight, a recoverable state, or a persistent state only when the corresponding rule is satisfied. The training study below retains only comparisons supported by the supplied study records; it does not establish a ranking against these recent defenses.

TEE-assisted trusted training. TEEs protect execution and key material and have been used to support FL integrity and attack mitigation [12], [13], [14], [15], [21]. Attestation can bind a measured environment to a reporting key; the signed report must additionally bind freshness and the submitted update. TTFL treats this authenticated reporting interface as a system assumption and evaluates the downstream controller separately.

3. System Boundary and Evidence Interface

The server receives an update ΔW_k and an authenticated report from client k , and applies $W^{(t)} = W^{(t-1)} + \Delta W_{global}^{(t)}$. The local-data factor is $p_k = n_k / \sum_j n_j$, where n_k counts the client’s training records. If a shared pool is replicated across clients, each local copy is counted in n_k . Filtering and clipping change the optimization trajectory, so sample-size weighting does not establish equivalence to the unfiltered objective over unique images.

The threat model permits poisoning of labels, inputs, objectives, and updates before report signing. The server follows the specified policy, sees individual updates, and holds a trusted proxy unavailable for client modification. Cryptographic secure aggregation is therefore outside this protocol. The reporting boundary in Fig. 1 protects a device key and binds identity, round, nonce, model digest, and update digest. GPU memory and sample semantics are

outside that boundary. Protection of a report does not make externally collected telemetry truthful. Trusted acquisition is an assumption on the reporting interface, not a property demonstrated by the controller tests. No platform-specific attestation or hardware timing result is claimed here.

The executable core accepts a valid-admission flag, normalized runtime entropy $h \in [0, 1]$, trainable update blocks, a server reference increment, and measured proxy loss and activation changes. It does not implement a TEE or a complete training application. A block is one named, floating-point trainable parameter tensor in a fixed manifest. Running statistics and integer buffers are excluded from both this manifest and the controller’s output; a training adapter must handle them separately without treating them as scored learning updates.

4. Evidence and State Control

Each round freezes the peer population, audits current updates, advances the states, and then aggregates eligible parameter blocks, as in Fig. 2. Let $\mathcal{D}_t$ contain valid submissions from clients not previously blacklisted. Before processing any state transition, freeze

$$\mathcal{R}_t = \{k \in \mathcal{D}_t : s_k^{(t-1)} \in \{\text{NORMAL}, \text{SUSPECT}\}, r_k^{(t-1)} < \tau_s\}. \quad (1)$$

A quarantined client supplies recovery evidence but never enters this round’s peer statistics. Here $\tau_s = 0.64$ is the review-risk threshold. Peer comparisons exclude the evaluated client itself.

4.1. Admission and Runtime Reliability

Let $M_k \in \{0, 1\}$ be the admission-verification result. Invalid admission excludes the update and freezes its temporal state. For valid nonzero submissions with complete server evidence, the runtime filter uses

$$R_k = \exp(\lambda h_k), \quad (2)$$

$$Z_k = w_a + (1 - w_a)(1 - h_k), \quad (3)$$

$$P_k^- = P_k + Q, \quad K_k = P_k^- / (P_k^- + R_k), \quad (4)$$

$$T_k' = T_k + K_k(Z_k - T_k), \quad (5)$$

$$P_k' = (1 - K_k)P_k^-, \quad (6)$$

$$\text{TrustScore}_k = M_k T_k' / (1 + P_k'). \quad (7)$$

The fixed controller parameters are $w_a = 0.7$, $\lambda = 1$, $Q = 0.01$, $T_0 = 1$, and $P_0 = 0.25$. Entropy increases measurement uncertainty and reduces the observation mean; it is not a label of maliciousness. Tests supply h directly, so their results do not validate a particular telemetry histogram, instruction-counter collector, or TEE deployment.

4.2. Server Audit and Optional Peer Evidence

The mandatory audit uses a server reference and proxy measurements. For a training adapter, the reference increment is one full-proxy cross-entropy SGD step, $g =$

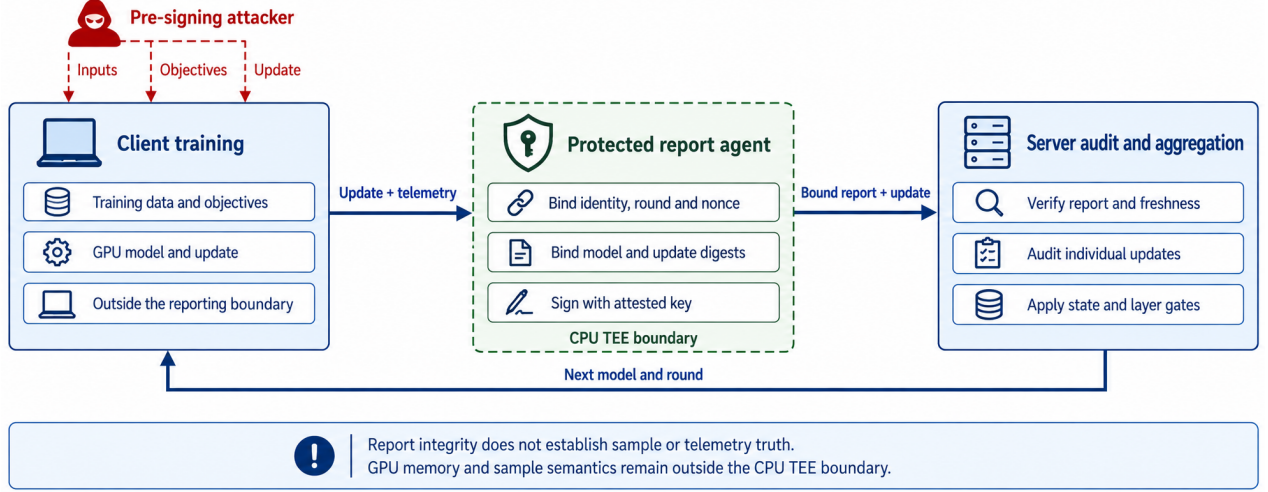


Figure 1. System roles and assumed reporting boundary. Red dashed paths locate attacks on inputs, objectives, or updates before signing. The protected agent binds identity, freshness, and model/update digests; the server audits individual updates before aggregation. The protected process authenticates reports, while content safety remains a server-audit responsibility.

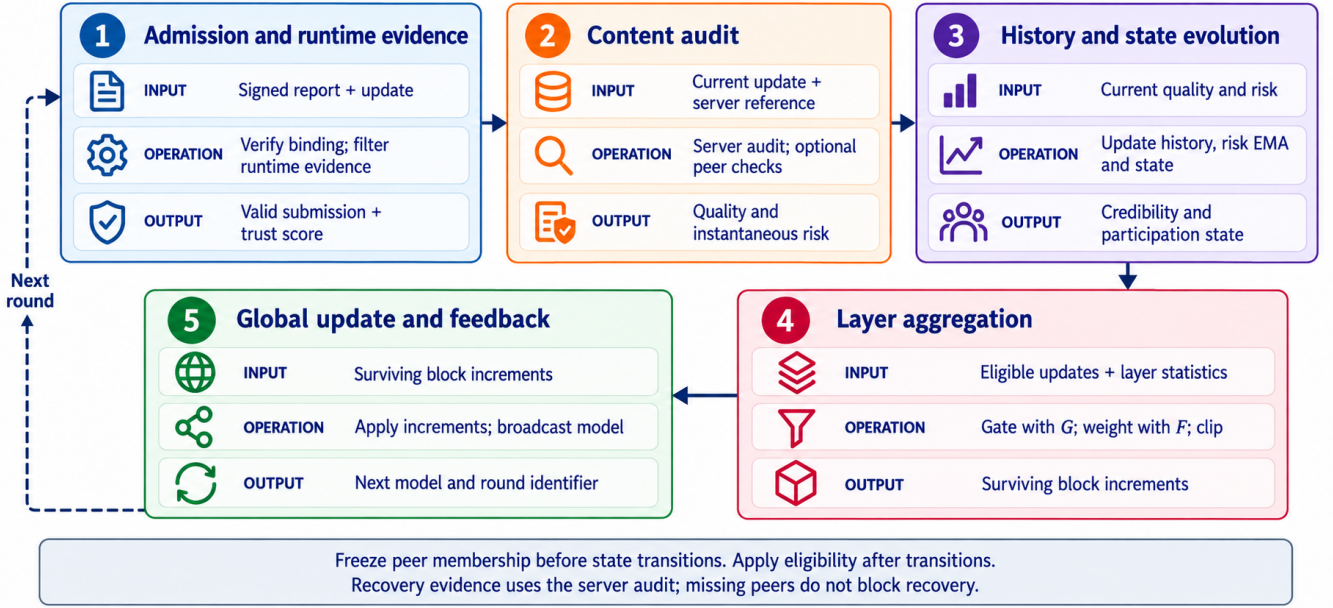


Figure 2. Five-stage trust-flow loop. The peer set is frozen before state transitions, while aggregation eligibility uses the updated state. A client starting in QUARANTINE may provide server-audited recovery evidence without entering that round’s peer statistics. Recovery to SUSPECT restores conditional aggregation eligibility in the same round; Section 4.3 specifies the transition rules.

$-10^{-3}\nabla_W \mathcal{L}_{proxy}(W)$, without momentum or weight decay. The controller consumes the resulting vector. Deterministic tests supply an explicit reference vector instead of running a model optimizer.

A cosine is available only when both vector norms exceed $\varepsilon_n = 10^{-12}$. An unavailable cosine is omitted from risk fusion; it is not set to zero and interpreted as disagreement. Let c_k denote the available reference cosine. The direction risk is $(1 - c_k)/2$ and the reference quality is $v_k = 0.6 + 0.4c_k$. When the reference direction is unavail-

able, proxy behavior defines $v_k = (1 - r_{loss,k})(1 - r_{act,k})$. For usable peers $j \in \mathcal{R}_t \setminus \{k\}$, positive trust weights produce a mean cosine $\bar{c}_k$ and $S_{peer,k} = 0.6 + 0.4\bar{c}_k$. Then

$$q_k = \begin{cases} 5S_{peer,k}v_k/(4S_{peer,k} + v_k), & \text{usable peers exist,} \\ v_k, & \text{otherwise.} \end{cases} \quad (8)$$

All available quantities lie in $[0, 1]$; no peer value is imputed when the set is empty. The peer-risk channel, when available, is $r_{peer,k} = \text{clip}(1 - \bar{c}_k, 0, 1)$.

Write $\Delta\mathcal{L}_k = \mathcal{L}_{proxy}(W + \Delta W_k) - \mathcal{L}_{proxy}(W)$. The mandatory server channels are

$$r_{loss,k} = \text{clip}\left(\frac{\max\{0, \Delta\mathcal{L}_k\}}{\delta_{loss}}, 0, 1\right), \quad (9)$$

$$r_{act,k} = \text{clip}(d_{act,k}/\delta_{act}, 0, 1). \quad (10)$$

For a batch of B matched inputs and activation vectors A_i at a declared representation, activation drift is

$$d_{act,k} = \frac{1}{B} \sum_{i=1}^B \text{KL}(\text{softmax}(A_{base,i}) \parallel \text{softmax}(A_{k,i})). \quad (11)$$

Softmax is taken over flattened features of each sample, never over the batch. The same inputs and evaluation mode are used before and after the candidate update. For a ResNet-18 adapter, the declared representation is the 512-vector after global average pooling. The array implementation tests this sample axis; it does not claim a trained ResNet measurement.

Each scale uses trusted calibration observations of its own nonnegative quantity: $\delta = \max\{10^{-6}, \text{median}(z) + 3 \text{MAD}(z)\}$. Calibration inputs are separate from final evaluation inputs. Positive scales are required inputs to the controller. Recovery fixtures use synthetic zero loss/activation changes and scales of one; those are test inputs, not fitted CIFAR-10 calibration values.

Amplitude risk is $\text{clip}(\|\Delta W_k\|/m_{\text{norm},k} - 1, 0, 1)$, where $m_{\text{norm},k}$ is the median nonzero norm of usable peer updates. With no usable peer, $m_{\text{norm},k} = \|g\|$. The channel is omitted if this scale is at most ε_n . Instantaneous risk z_k is the maximum of the two mandatory server channels and all available geometric channels. Missing proxy evidence freezes history, risk, and counters and yields zero aggregation weight. A full-update norm at most ε_n is a no-operation submission with the same freeze rule; it cannot build reputation or create a direction alarm.

This separates recovery from peer availability: when all clients are quarantined, valid nonzero updates can still be evaluated against the trusted server. Loss of the server audit service intentionally suspends recovery; that external dependency is distinct from a circular peer-state dependency.

4.3. History, Risk, and Persistent Transitions

Figure 3 connects the four participation states using explicit persistence conditions. Historical utility and short-term risk jointly determine review, while the starting state determines the available transition. BLACKLIST is enforced by the server rejecting future submissions.

Each complete nonzero submission adds unit evidence to the history model:

$$a'_k = \beta_h a_k + q_k, \quad b'_k = \beta_h b_k + 1 - q_k, \quad (12)$$

$$H'_k = a'_k / (a'_k + b'_k), \quad r'_k = \beta_r r_k + (1 - \beta_r) z_k. \quad (13)$$

Initialization is $a_0 = b_0 = 1$, $r_0 = 0$; $\beta_h = 0.9$ and $\beta_r = 0.85$. The strictly positive evidence masses require no additive denominator offset. History and risk share some

geometric inputs and are not statistically independent evidence streams.

Utility threshold. The review condition is

$$B_k = (H'_k < 0.26) \vee (r'_k \geq 0.64). \quad (14)$$

Thus 0.26 is a lower bound on historical utility, not a threshold on $1 - H_k$. A favorable recovery observation satisfies $\neg B_k$. Transitions are evaluated using the starting state and updated history/risk. At most one transition occurs per complete submission; escalation has priority. Each counter counts consecutive qualifying observations in its starting state, resets on a failed condition, and all counters reset after a transition. Missing or no-operation submissions freeze counters. There is no instantaneous override of the EMA or extra attack-specific blacklist branch.

From NORMAL, two consecutive B_k observations lead to SUSPECT. From SUSPECT, three consecutive B_k observations or a current $z_k \geq 0.80$ lead to QUARANTINE; two consecutive $\neg B_k$ observations restore NORMAL. From QUARANTINE, four consecutive $r'_k \geq 0.90$ observations lead to BLACKLIST; otherwise, two consecutive $\neg B_k$ observations restore SUSPECT.

Newly quarantined or blacklisted updates are excluded immediately. BLACKLIST also rejects future submissions. Recovery to SUSPECT restores conditional current-round eligibility. Recovery does not itself imply passing a layer gate or applying a nonzero model update; those endpoints are logged separately.

5. Aggregation with Attainable Gates

Figure 4 separates the evidence inputs, layer decisions, and constrained execution. The normalized score G_k determines gate passage, whereas the raw credibility F_k determines each survivor's relative weight.

Let $\mathcal{E}_t$ contain complete nonzero audited submissions. After state transitions, form $\mathcal{A}_t = \{k \in \mathcal{E}_t : s_k^{(t)} \in \{\text{NORMAL}, \text{SUSPECT}\}\}$. The unnormalized credibility score remains

$$F_k = \text{TrustScore}_k^3 q_k \sqrt{H'_k (1 - r'_k)}. \quad (15)$$

A fixed gate near one is unsuitable because filter uncertainty suppresses the entire attainable score range. Instead, normalize only the gate comparison by a deterministic best-evidence envelope at the client's evidence age n_k^e . Here n_k^e counts complete nonzero submissions and is distinct from the training-record count n_k .

Starting from $\underline{P}_0 = 0.25$, $a_0^+ = b_0^- = 1$, define

$$\underline{P}_n = \frac{\underline{P}_{n-1} + Q}{1 + \underline{P}_{n-1} + Q}, \quad (16)$$

$$a_n^+ = 0.9a_{n-1}^+ + 1, \quad b_n^- = 0.9b_{n-1}^-, \quad (17)$$

$$U_n = (1 + \underline{P}_n)^{-3} \sqrt{a_n^+ / (a_n^+ + b_n^-)}. \quad (18)$$

Since $R \geq 1$, $T \leq 1$, $q \leq 1$, and $r \geq 0$, induction gives $F_k \leq U_{n_k^e}$. The bound is attained with valid admission,

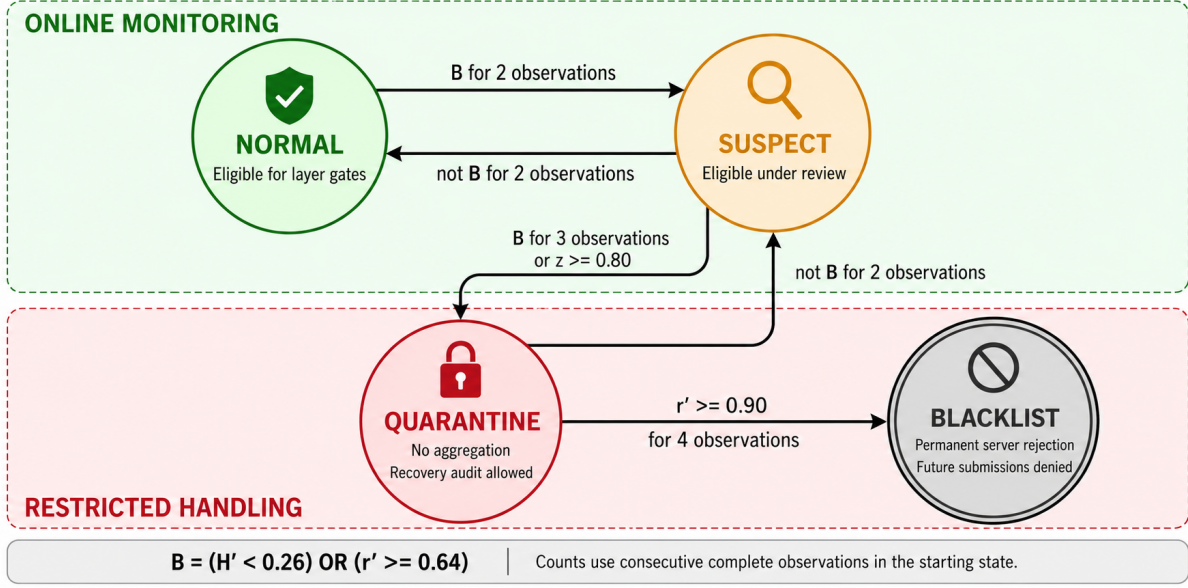


Figure 3. Four-state participation policy with explicit transition conditions. $B_k = (H'_k < 0.26) \vee (r'_k \geq 0.64)$; the diagram suppresses the client subscript. Counts refer to consecutive complete observations in the starting state. Escalation takes priority, and BLACKLIST is terminal.

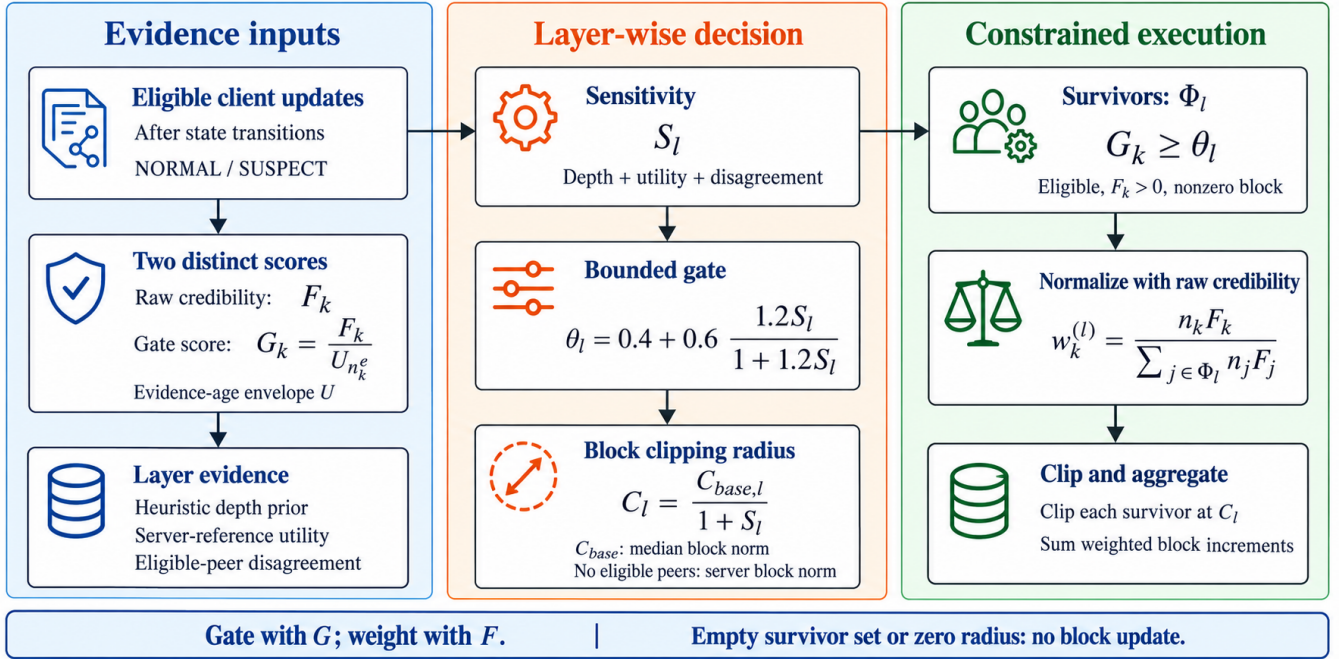


Figure 4. Layer-wise aggregation after state transitions. Only eligible NORMAL/SUSPECT submissions can survive a gate. The comparison uses $G_k = F_k / U_{n_k^e}$, but weights retain $n_k F_k$. Depth is a heuristic prior; clipping uses a separate scale for each parameter block. An empty survivor set or a radius at most ε_n produces no block update, without bypassing state exclusion.

$h = 0$, $q = 1$, and $z = 0$ at every complete observation. The dimensionless gate score is $G_k = F_k / U_{n_k^e}$; actual weights continue to use F_k , preserving the covariance penalty there.

5.1. Layer Statistics and Clipping

Layer statistics use $\mathcal{T}_t = \mathcal{R}_t \cap \mathcal{A}_t$. This is a current-eligibility filter on the frozen peer set. Newly recovered clients may aggregate but cannot influence the same-round

layer statistics. For blocks $l = 1, \dots, L$, define

$$S_{depth}^{(l)} = e^{-3l/L}, \quad (19)$$

$$S_{util}^{(l)} = \|g^{(l)}\| / \max_m \|g^{(m)}\|, \quad (20)$$

$$S_{sec}^{(l)} = \frac{1}{2}(1 - \text{mean}_{k \in \mathcal{T}_l} \cos(\Delta W_k^{(l)}, g^{(l)})), \quad (21)$$

$$S_l = 0.3S_{depth}^{(l)} + 0.3(1 - S_{util}^{(l)}) + 0.4S_{sec}^{(l)}. \quad (22)$$

Only available cosines enter the mean. An empty cosine set gives no peer disagreement contribution, $S_{sec} = 0$, and is recorded as missing peer evidence. If all root blocks have zero norm, set $S_{util} = 0$. The depth component is a heuristic prior, not a measured privacy risk or a privacy guarantee.

The bounded gate and clipping radius are

$$\theta_l = 0.4 + 0.6 \frac{1.2S_l}{1 + 1.2S_l}, \quad C_l = \frac{C_{base,l}}{1 + S_l}. \quad (23)$$

Here $C_{base,l}$ is the median norm of block l over $\mathcal{T}_l$; it is not the norm of the complete model update. If $\mathcal{T}_l$ is empty, use the corresponding server-reference norm $\|g^{(l)}\|$. A radius at most ε_n leaves that block unchanged. Thus trusted server evidence also supplies a defined aggregation path after collective quarantine, without using quarantined updates to set their own thresholds or clipping radii.

Because $0 \leq S_l \leq 1$, $0.4 \leq \theta_l \leq 0.727273 < 1$. The attainable best-evidence score $G_k = 1$ passes every such gate, including at the first complete submission. This eliminates structural impossibility caused by the score envelope; it does not guarantee that noisy or heterogeneous updates pass.

5.2. Survivors and Applied Updates

For a positive clipping radius, survivors are

$$\Phi_l = \{k \in \mathcal{A}_t : G_k \geq \theta_l, F_k > 0, \|\Delta W_k^{(l)}\| > \varepsilon_n\}. \quad (24)$$

If Φ_l is empty, $\Delta W_{global}^{(l)} = 0$. No state exclusion or gate is bypassed by a fallback. Otherwise positive survivor mass permits exact normalization:

$$w_k^{(l)} = \frac{n_k F_k}{\sum_{j \in \Phi_l} n_j F_j}, \quad (25)$$

$$\widehat{\Delta W}_k^{(l)} = \Delta W_k^{(l)} \min\{1, C_l / \|\Delta W_k^{(l)}\|\}, \quad (26)$$

$$\Delta W_{global}^{(l)} = \sum_{k \in \Phi_l} w_k^{(l)} \widehat{\Delta W}_k^{(l)}. \quad (27)$$

Weights sum to one on a nonempty survivor set. Each clipped block norm is at most C_l , so the triangle inequality also bounds the aggregate norm by C_l . Individual nonzero contributions can cancel; a selected block, a positive weight, and a nonzero aggregate increment are therefore distinct endpoints.

The audit records client, round, trainable-block name, starting/ending state, quality, instantaneous and smoothed risk, history, persistence counters, transition reason, raw

and gate scores, threshold, survivor membership, normalized weight, pre/post-clipping norms, and applied contribution norm. Per-block logs additionally record survivor count, clipping radius, skipped updates, and the norm of the summed increment.

6. Training Study

6.1. Protocol, Provenance, and Metrics

The CIFAR-10 results in this section concern the original training implementation. The author-confirmed protocol uses five matched seed, partition, and attack schedules on each of Intel and AMD platforms, giving ten platform-seed runs. Table 1 records this study's settings; it does not infer training settings from the controller fixtures. Admission tests additionally used five Sybil and three free-rider identities that were rejected before the twenty-client training statistics were collected. The supplied figures are confirmed by the authors as measurements, but their individual platform/seed identifiers and the complete aggregation script are not available in the current reproducibility bundle. The study summary and representative curves therefore have distinct statistical roles. The model and partition manifest associated with each retained curve is also not independently linked to Table 1; the curves provide descriptive training evidence rather than a newly matched comparison. Section 7 separately checks the explicit controller; no unchanged training behavior is inferred for its recovery, gate, or clipping refinements.

The trigger attack modifies 20% of local samples with a bottom-right 3×3 patch and target class 0. The clean-label and semantic attacks also use 20% subsets and target class 0; their perturbation budget and semantic attribute are not recoverable from the supplied study description. The five matched schedules control repeated data/attack randomization across platforms; ten platform-seed observations do not imply ten independent partitions. The shared pool is 10% of unique training images, not 10% of each client's local dataset. Its local fraction is $5000/(5000 + n_k^{\text{exclusive}})$. The Dirichlet parameters describe the exclusive portions; a shared component remains present in every client.

Clean accuracy is the fraction of correctly classified evaluation images. For targeted attack a , the reported definition is $\text{ASR}_a = |\{x \in \mathcal{B}_a : \arg \max f(x) = y_a^*\}|/|\mathcal{B}_a|$, and the study reports the macro-average over trigger, clean-label, and semantic attacks. The underlying per-attack counts and target-class exclusion rule are not supplied, so these values cannot be recomputed under a harmonized ASR protocol. Untargeted label/sign/scaling attacks are not terms in that average. Permanent-ban FPR counts benign clients ending in BLACKLIST, not temporary review or quarantine. Because the proxy originates from the test-source split, independence from final evaluation requires an index-level check that is not available here; held-out generalization is not established by this provenance.

TABLE 1. REPORTED TRAINING PROTOCOL. MISSING RUN-LEVEL DETAILS ARE STATED EXPLICITLY; CONTROLLER-TEST PARAMETERS DO NOT FILL THOSE GAPS.

Item	Training-study setting
Model	CIFAR-10 ResNet-18: 3×3 stride-1 stem, no initial max pool, ten outputs; initialization checkpoint/seed list unavailable.
Clients	20 admitted clients; 14 benign and 6 malicious.
Partition	45,000 exclusive training images: clients 0–9 IID, 10–14 Dirichlet $\alpha = 1$, 15–19 $\alpha = 0.1$.
Shared pool	5,000 training images copied to every client, in addition to its exclusive partition; each local copy counts in n_k .
Server proxy	500 balanced CIFAR-10 test-source images (50/class), excluded from client training; exact indices unavailable.
Attacks	C0 label flipping (0.5); C1 trigger, C2 clean-label, C3 semantic; C4 sign flipping; C5 100× scaling.
Optimizer	SGD, learning rate 0.001, momentum 0.9; three local epochs; 30 rounds for the main study, 50 for aggregation comparison.
Repetitions	Intel and AMD, five matched schedules each; ten-run reported mean and SD, without a platform-effect test.
Evaluation	Clean accuracy and reported targeted ASR; final-test and calibration indices, overlap audit, and per-attack counts unavailable.

6.2. Main Performance and Client Behavior

Table 2 retains the author-confirmed TTFL summary from the original manuscript. It is a reported aggregate, not a recomputation from partial local logs. Baseline rows without a complete matching run record are omitted from this table, so it supports no new method-ranking or significance claim. The zero permanent-ban FPR describes the tested benign clients; it does not imply that all their updates passed every layer gate.

TABLE 2. REPORTED TTFL TRAINING SUMMARY UNDER 30% MIXED ATTACKS. ACCURACY AND ASR ARE THE REPORTED MEAN $\pm$ SD OVER TEN PLATFORM-SEED RUNS; THE PER-ATTACK ASR AUDIT REMAINS INCOMPLETE.

Metric	Reported value (%)
Clean accuracy	92.31 $\pm$ 0.09
Targeted ASR (reported macro-average)	10.21 $\pm$ 0.05
Benign permanent-ban FPR	0.00

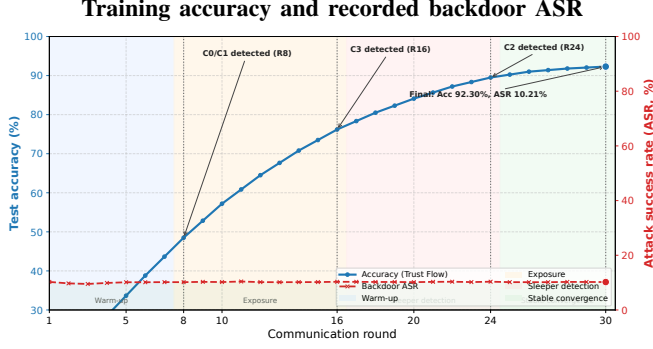
Figure 5(a) shows the measured accuracy and recorded backdoor-ASR trajectories. The red series is an attack-success measurement, not a fixed reference line. Its mapping to the three per-attack outcomes is not documented, so it is not identified with the macro-average at every round. The displayed accuracy endpoint is 92.30%, whereas Table 2 reports the repeated-run mean of 92.31%; the curve is not used to recover that mean or its SD. Figure 5(b) shows distinct historical-utility and risk trajectories. Its detection,

interception, and isolation annotations retain their original meanings and are not converted into BLACKLIST times for the executable controller. Together the panels describe learning and client behavior without equating temporary restriction with permanent removal.

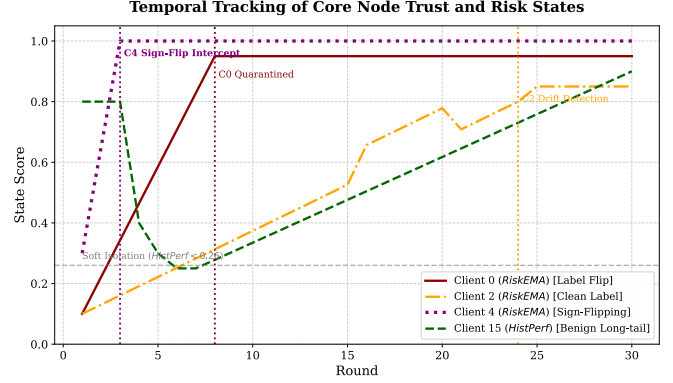
6.3. Aggregation Variants and Data Partitions

Figure 6(a) compares three operations over 50 rounds. Global-Clipping applies a common clipping policy; hierarchical gating without renormalization removes disallowed layer contributions without redistributing their weights; the full method normalizes weights over each layer’s survivors. The displayed curves also differ in gating and clipping, so the full performance gap cannot be attributed solely to normalization. The 30-round marker is a within-curve reference, not the end of this comparison.

Figure 6(b) compares the reported Dirichlet settings. The TTFL series changes less across the displayed settings than Krum and FLTrust. The shared pool remains part of the training protocol; α describes the exclusive data rather than the full local distribution. The sweep’s exact per-client allocation manifests and repeated-run summaries are unavailable, so the plot does not establish distribution-independent robustness or isolate the effect of the shared pool.

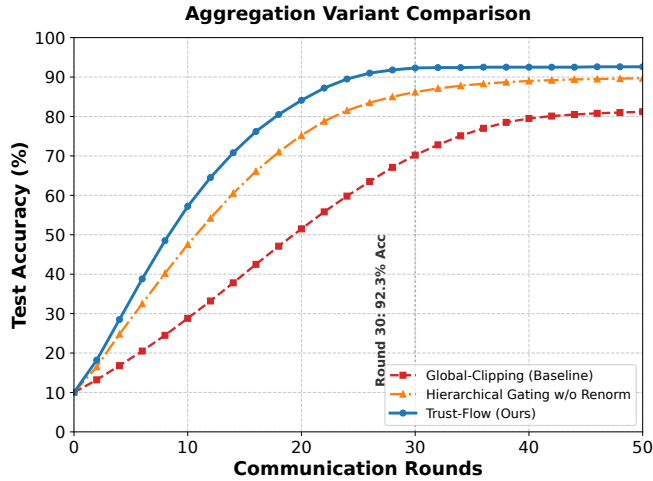


(a) Accuracy and measured attack-success trajectory.

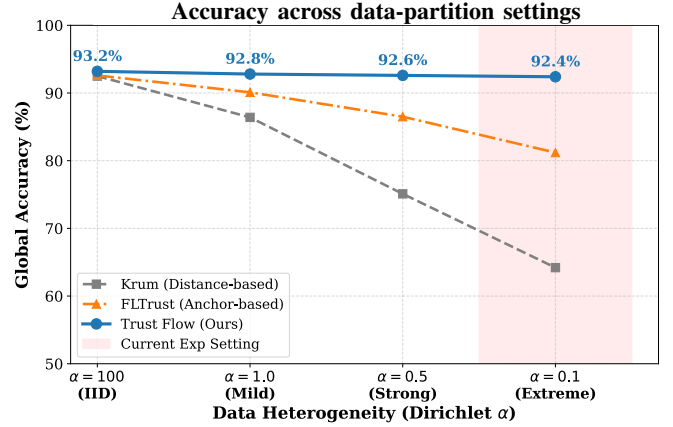


(b) Client historical utility and risk.

Figure 5. CIFAR-10 training behavior: (a) accuracy and recorded backdoor ASR over 30 rounds; (b) representative client trajectories with detection and restriction events. Curves describe the training implementation, not the constructed controller tests.



(a) Aggregation variants over 50 rounds.



(b) Accuracy at the reported Dirichlet settings.

Figure 6. Training-study comparisons. (a) Global clipping, hierarchical gating without weight renormalization, and full TTFL; the comparison changes multiple aggregation operations. (b) Accuracy across exclusive-data partition settings in the presence of a shared training pool. No unreported uncertainty estimates are inferred from either panel.

7. Executable Protocol Verification

7.1. Scope and Reproduction

The controller in `protocol/controller.py` implements the specified audit, temporal states, and aggregation; `config.json` fixes its parameters. Eleven test groups export state, block, and layer events with source hashes. These are constructed control tests, not classifier training. The randomized check uses seed 20261001 and 100 sequences of 100 observations. Table 3 in the appendix lists the conditions and endpoints.

The partial training-code snapshot differs in its history update, raw-score gating, clipping scales, and weighting. It is not the complete implementation underlying Section 6, so a matched training comparison is still needed to quantify the effect of the explicit controller rules.

7.2. Recovery and Startup Behavior

A counterexample combines the deliberately incorrect cutoff $H < 0.74$ with mandatory peers: $q = 0.8$, $z = 0.3$ lead to quarantine at observation five and frozen history through forty. This does not describe the training implementation, whose state annotation already uses $H < 0.26$. With that utility floor, the constructed startup sequence remains NORMAL.

The matched recovery test instead initializes twenty clients in QUARANTINE with risk 0.95, history 0.5, and reset counters. Each submits the server-aligned update $(1, 0)$ with zero loss/activation change, unit calibration scales, and zero entropy. Thus $q = 1$, $z = 0$ despite an empty peer set. Risk falls below 0.64 at audit three; the next favorable observation restores SUSPECT and nonzero aggregation at four. All clients return to NORMAL at six (Fig. 7). In the matched arm, only the requirement for usable peers before

state advancement changes. All clients remain quarantined and aggregation remains zero through ten attempted audits. This isolates the peer dependency under favorable server evidence; it is not a training comparison or a universal guarantee of benign recovery.

7.3. Risk Timing and Gate Reachability

With $r_0 = 0$ and $z_t \leq 1$, pure EMA satisfies $r_t \leq 1 - 0.85^t$. At $t = 14$ its upper bound is 0.89723, below the blacklist threshold; the first crossing is at $t = 15$. Four consecutive qualifying observations while quarantined therefore cannot finish before $t = 18$. The maximal-risk test matches that bound and rejects the next submission at $t = 19$.

An unscaled gate $0.4 + 1.2S_l$ can exceed the first-observation raw-score ceiling 0.469222, or the asymptotic ceiling of about 0.761393. The normalized comparison instead attains one under ideal evidence at every age, while its maximum gate is 0.727273. The 10,100 age-sensitivity checks verify these formulas. They establish gate reachability, not an acceptable survivor rate on trained models.

8. Limitations and Conclusion

TTFL combines authenticated reports, current-update audits, separate utility and risk streams, and layer-wise aggregation. The training study characterizes model and client behavior under mixed attacks; the controller checks establish conditional recovery, attainable gates, and bounded normalized aggregation. The matched constructed test shows why recovery audits must remain available when eligible peers disappear.

The training summary does not measure the effect of these controller refinements. That requires matched training, run-level configuration identities, per-attack counts, and disjoint proxy/calibration/evaluation indices. The shared pool also limits the heterogeneity analysis. The partial code cannot establish full implementation correspondence. Hardware attestation performance, general Byzantine robustness, and superiority over a tuned single-stream policy remain outside the verified claims.

Appendix A. Supplementary Controller Checks

A.1. Constructed Recovery Trace

Figure 7 shows the fixed-input controller test described in Section 7. All clients submit the same two-dimensional update at every audit. The discrete state steps and constant applied-update norm after recovery follow from those test inputs; they are not classifier learning curves.

TABLE 3. CONSTRUCTED CONTROLLER CHECKS AND VERIFIED ENDPOINTS.

Case	Condition and checked endpoint
Cutoff/peers	$q = 0.8, z = 0.3$, cutoff 0.74, mandatory peers: SUSPECT at 2, QUARANTINE at 5; frozen through 40.
Healthy startup	Same q, z , utility floor 0.26: NORMAL through 40 complete observations.
Collective recovery	20 quarantined clients, $r_0 = 0.95$: server audit restores aggregation at 4, NORMAL at 6; matched mandatory-peer arm stays frozen with zero aggregate through 10.
Maximal risk	$q = z = 1, r_0 = 0$: SUSPECT at 8, QUARANTINE at 9, BLACKLIST at 18.
Gate envelope	Ideal evidence at 100 ages and 101 sensitivities: all 10,100 gate checks pass.
Empty survivors	Low-score candidate plus high-score quarantined client: zero increment, no backfill.
Missing evidence	Missing proxy freezes state; server audit with one client gives sole-survivor weight 1.
Zero/tiny updates	Norms $0, 10^{-15}, 10^{-12}$: no direction alarm, history gain, or applied update.
Weights/ clipping	Equal scores, sample counts 1 : 3; weights 0.25/0.75; aggregate norm bounded by the radius.
Calibration	Zero calibration inputs: positive 10^{-6} floor; two-sample KL uses feature-wise softmax.
Random inputs	100 sequences of 100 observations: bounded states/scores; excluded states have no eligibility.

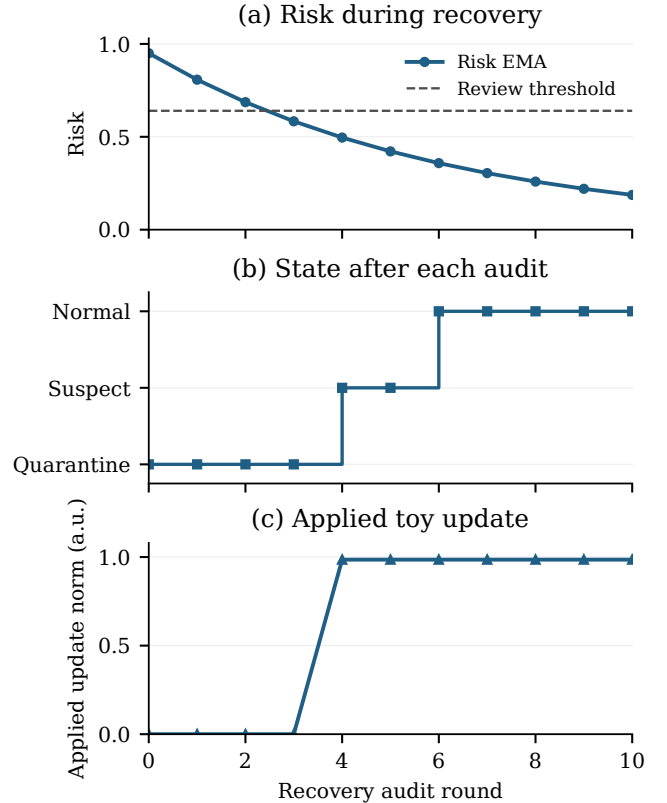


Figure 7. Constructed collective recovery: common risk/state trajectory for twenty clients and applied two-dimensional update norm in arbitrary units.

References

- [1] P. Kairouz, H. B. McMahan, B. Avent *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agueray Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, vol. 54, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [3] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/6617-machine-learning-with-adversaries-byzantine-tolerant-gradient-descent>
- [4] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, 2018, pp. 5650–5659. [Online]. Available: <https://proceedings.mlr.press/v80/yin18a.html>
- [5] X. Cao, M. Fang, J. Liu, and N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.
- [6] C. Fung, C. J. M. Yoon, and I. Beschastnikh, “The limitations of federated learning in Sybil settings,” in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 301–316. [Online]. Available: <https://www.usenix.org/conference/raid2020/presentation/fung>
- [7] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, “FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2545–2555.
- [8] P. Rieger, T. D. Nguyen, M. Miettinen, and A. R. Sadeghi, “Deep-Sight: Mitigating backdoor attacks in federated learning through deep model inspection,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.
- [9] T. D. Nguyen, P. Rieger, H. Chen, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, S. Zeitouni, F. Koushanfar, A.-R. Sadeghi, and T. Schneider, “FLAME: Taming backdoors in federated learning,” in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 1415–1432. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/nguyen>
- [10] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A. R. Sadeghi, “CrowdGuard: Federated backdoor detection in federated learning,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.
- [11] H. Yang, W. Xi, Y. Shen, C. Wu, and J. Zhao, “RoseAgg: Robust defense against targeted collusion attacks in federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2951–2966, 2024.
- [12] Y. Chen, F. Luo, T. Li, T. Xiang, Z. Liu, and J. Li, “A training-integrity privacy-preserving federated learning scheme with trusted execution environment,” *Information Sciences*, vol. 522, pp. 69–79, 2020.
- [13] T. Xu, K. Zhu, A. Andrzejak, and L. Zhang, “Distributed learning in trusted execution environment: A case study of federated learning in SGX,” in *Proceedings of the 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC)*, 2021, pp. 450–454.
- [14] S. Queyruet, V. Schiavoni, and P. Felber, “Mitigating adversarial attacks in federated learning with trusted execution environments,” in *Proceedings of the IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, 2023, pp. 626–637.
- [15] Z. Lu, L. Wang, Z. Zhang, M. Huang, J. Wang, and M. Li, “TMT-FL: Enabling trustworthy model training of federated learning with malicious participants,” *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 3, pp. 2723–2740, 2025.
- [16] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How To Backdoor Federated Learning,” in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, vol. 108, 2020, pp. 2938–2948. [Online]. Available: <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
- [17] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, “Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks,” in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 707–723.
- [18] J. Zhang, C. Zhu, X. Sun, C. Ge, B. Chen, W. Susilo, and S. Yu, “FLPurifier: Backdoor defense in federated learning via decoupled contrastive training,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4752–4766, 2024.
- [19] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, “ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1639–1654, 2022.
- [20] L. Wang, M. Huang, Z. Zhang, M. Li, J. Wang, and K. Gai, “RaSA: Robust and adaptive secure aggregation for edge-assisted hierarchical federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4280–4295, 2025.
- [21] X. Zhang, Z. Chen, G. Chen, X. Feng, Q. Shen, and Z. Wu, “RPPFL: Robust and privacy-preserving federated learning via trusted execution environments,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.